SYNAPSE API SDK Guide

SYNAPSE Development Team

March 2026

Contents

1 Introduction **5**
1.1 Overview . . . 5
1.2 Authentication Overview . . . 5
1.3 Base URL and Versioning . . . 5
1.4 Rate Limits . . . 6
1.5 Response Format . . . 6
1.6 Error Codes Reference . . . 6
1.7 SDK Availability . . . 7

2 Python SDK **8**
2.1 Installation . . . 8
2.2 Client Initialization . . . 8
2.3 Authentication . . . 8
2.3.1 Token-based Authentication . . . 8
2.3.2 API Key Authentication . . . 9
2.3.3 Token Auto-Refresh . . . 9
2.4 Articles API . . . 9
2.4.1 List Articles . . . 9
2.4.2 Get Article . . . 10
2.4.3 Search Articles . . . 10
2.4.4 Bookmark Article . . . 10
2.5 Papers API . . . 10
2.5.1 List Papers . . . 10
2.5.2 Get Paper . . . 11
2.6 Repositories API . . . 11
2.6.1 Trending Repositories . . . 11
2.7 AI Chat API . . . 11
2.7.1 Send Chat Message . . . 11
2.7.2 Summarize Content . . . 12
2.8 Agents API . . . 12
2.8.1 Create Task . . . 12
2.8.2 Get Task Status . . . 12
2.8.3 Generate Document . . . 13
2.9 Workflows API . . . 13
2.9.1 Create Workflow . . . 13
2.9.2 List Workflows . . . 14
2.9.3 Trigger Workflow . . . 14
2.10 Trends API . . . 14
2.10.1 Get Technology Radar . . . 14
2.10.2 Get Timeline . . . 14
2.11 Async Client . . . 15
2.11.1 Concurrent Requests . . . 15

2.12 Pagination Helper . . . 15
2.13 Error Handling . . . 15
2.13.1 Exception Classes . . . 15
2.13.2 Retry Logic . . . 16
2.14 Webhook Handler . . . 16

3 JavaScript/TypeScript SDK **18**
3.1 Installation . . . 18
3.2 TypeScript Types . . . 18
3.3 Client Initialization . . . 19
3.4 Articles API . . . 20
3.4.1 List Articles . . . 20
3.4.2 Get Article . . . 20
3.4.3 Search Articles . . . 20
3.4.4 Bookmark Article . . . 21
3.5 Papers API . . . 21
3.6 Repositories API . . . 21
3.7 AI Chat API . . . 21
3.8 Agents API . . . 22
3.9 Workflows API . . . 22
3.10 Trends API . . . 23
3.11 React Hooks . . . 23
3.11.1 useSynapseArticles . . . 23
3.11.2 useSynapseChat . . . 24
3.11.3 useSynapseTrends . . . 24
3.12 Error Handling . . . 24

4 Authentication Guide **26**
4.1 Register and Get API Key . . . 26
4.2 JWT Authentication Flow . . . 26
4.2.1 Login Endpoint . . . 26
4.2.2 Refresh Token Endpoint . . . 27
4.2.3 Python JWT Example . . . 27
4.3 API Key Authentication . . . 28
4.3.1 Python API Key Example . . . 28
4.4 Token Storage Best Practices . . . 28
4.4.1 Secure Token Storage (Python) . . . 29
4.5 Handling 401 Errors . . . 29
4.5.1 Automatic Token Refresh (Python) . . . 29

5 Webhooks **30**
5.1 Available Webhook Events . . . 30
5.2 Webhook Registration . . . 30
5.3 Webhook Payload Format . . . 31
5.4 Webhook Signature Verification . . . 31
5.5 Webhook Handler Example . . . 31
5.5.1 FastAPI Webhook Handler . . . 31
5.6 Retry Policy . . . 32

6 Real-Time Updates **33**
6.1 WebSocket Endpoint . . . 33
6.2 Server-Sent Events (SSE) Endpoint . . . 33
6.3 Available Channels . . . 33
6.4 Python SSE Client Example . . . 33
6.5 JavaScript EventSource Example . . . 34

6.6 AI Chat Streaming (SSE) . . . 34

7 Rate Limiting **36**
7.1 Rate Limit Headers . . . 36
7.2 Rate Limits by Endpoint . . . 36
7.3 Handling 429 Too Many Requests . . . 36
7.4 SDK Automatic Retry with Backoff . . . 37

8 Pagination **38**
8.1 Standard Pagination Format . . . 38
8.2 Cursor-based Pagination for Real-Time Feeds . . . 38
8.3 Python Pagination Helper . . . 39
8.4 JavaScript Pagination . . . 39

9 Integration Examples **41**
9.1 Example 1: Personal Tech Newsletter . . . 41
9.2 Example 2: Slack Bot for Tech Questions . . . 42
9.3 Example 3: Automated Weekly AI Research Report . . . 43
9.4 Example 4: GitHub Actions Release Notes Generator . . . 44
9.5 Example 5: React Component for Trending Technologies . . . 45

10 Semantic Search Guide **47**
10.1 How Semantic Search Works . . . 47
10.2 Semantic Search Endpoint . . . 47
10.3 Search Filters . . . 47
10.4 Search Result Ranking . . . 48
10.5 Best Practices for Query Formulation . . . 48
10.5.1 Examples . . . 48
10.6 Cross-Content-Type Search . . . 49

11 AI Agent API Guide **50**
11.1 Available Agent Task Types . . . 50
11.2 Async Task Execution Pattern . . . 50
11.2.1 Implementation . . . 50
11.3 Task Status Enum . . . 51
11.4 WebSocket Real-Time Agent Progress . . . 51
11.5 Generate Document Task . . . 52
11.6 Create Project Task . . . 52
11.7 Analyze Trends Task . . . 52

12 API Changelog **54**
12.1 Version 1.0.0 - March 2026 . . . 54
12.1.1 Features . . . 54
12.1.2 SDKs . . . 54
12.1.3 Endpoints . . . 55
12.2 Versioning Policy . . . 55
12.3 Deprecation Policy . . . 55
12.3.1 Deprecation Example . . . 55
12.4 Migration Guide Format . . . 56
12.4.1 Example Migration Guide . . . 56
12.5 Future Versions Roadmap . . . 57
12.5.1 v1.1 (Expected June 2026) . . . 57
12.5.2 v1.2 (Expected September 2026) . . . 57
12.5.3 v2.0 (Expected March 2027) . . . 57
12.6 Support and Feedback . . . 57

12.7 Rate Limit Status . . . 57
12.8 API Health and Monitoring . . . 58
12.9 API Performance Metrics . . . 58
12.10 Best Practices Summary . . . 58
12.10.1 API Usage . . . 58
12.10.2 Security . . . 59
12.10.3 Performance . . . 59

13 Appendix: Complete API Reference 60
13.1 Environment Variables . . . 60
13.2 Common Response Codes . . . 60
13.3 Request Header Reference . . . 60
13.4 Response Header Reference . . . 61
13.5 Useful Resources . . . 61
13.6 Code Examples Repository . . . 61

Chapter 1

Introduction

1.1 Overview

The SYNAPSE API is a comprehensive REST API built with Django and FastAPI that provides developers with programmatic access to the world's largest curated technology knowledge base. The API enables integration of SYNAPSE's powerful capabilities into custom applications, workflows, and automation systems.

SYNAPSE API offers the following core capabilities:

- Access to curated technology articles, research papers, and code repositories
- Semantic search across all content with advanced filtering capabilities
- AI-powered chat and content summarization
- Autonomous agent task execution for document generation and analysis
- Workflow automation and customization
- Real-time trend analysis and technology radar
- Webhook support for event-driven integrations
- WebSocket and Server-Sent Events (SSE) for real-time updates

1.2 Authentication Overview

SYNAPSE API uses JWT (JSON Web Token) Bearer token authentication as the primary authentication method. Developers can authenticate using:

1. API Key: A long-lived key issued for server-to-server communication
2. JWT Bearer Token: Short-lived access tokens obtained through the login flow

All API requests must include the authentication token in the Authorization header:

```
Authorization: Bearer YOUR_ACCESS_TOKEN
```

1.3 Base URL and Versioning

The base URL for all API requests is:

```
https://api.synapse.ai/api/v1/
```

SYNAPSE follows semantic versioning. The current version is v1, which is reflected in all endpoint URLs. Future major version changes will be available at /api/v2/, /api/v3/, etc.

1.4 Rate Limits

The SYNAPSE API implements rate limiting based on subscription tier:

Tier	Rate Limit	Burst Allowance
Free	100 requests/minute	150 requests/minute
Pro	1000 requests/minute	1500 requests/minute
Enterprise	Unlimited	Unlimited

Rate limits are reset on a rolling basis every 60 seconds. Burst allowances permit temporary traffic spikes beyond the stated limit.

1.5 Response Format

All API responses follow a standardized JSON envelope format:

```
{
  "success": true,
  "data": { /* response data */ },
  "error": null,
  "meta": {
    "timestamp": "2026-03-15T10:30:45Z",
    "request_id": "req_abc123xyz",
    "version": "1.0.0"
  }
}
```

For successful requests, the `success` field is `true`, data is populated, and `error` is `null`. For failed requests, `success` is `false`, `data` is `null`, and `error` contains error details.

1.6 Error Codes Reference

The following table lists standard error codes returned by the SYNAPSE API:

Code	HTTP Status	Description
INVALID_REQUEST	400	Request validation failed
UNAUTHORIZED	401	Missing or invalid authentication token
FORBIDDEN	403	Insufficient permissions for resource
NOT_FOUND	404	Resource not found
CONFLICT	409	Resource conflict (e.g., duplicate)
RATE_LIMITED	429	Request rate limit exceeded
INTERNAL_ERROR	500	Internal server error
SERVICE_UNAVAILABLE	503	Service temporarily unavailable
AUTH_EXPIRED	401	Authentication token expired
AUTH_INVALID	401	Authentication token invalid or malformed
VALIDATION_ERROR	400	Request parameters validation failed
NOT_IMPLEMENTED	501	Feature not yet implemented

1.7 SDK Availability

Official SYNAPSE SDKs are available for the following platforms:

1. Python 3.8+ - `synapse-sdk` package
2. JavaScript/TypeScript - `@synapse-ai/sdk` package

These SDKs provide type safety, automatic error handling, built-in retry logic, and convenient methods for all API endpoints. SDKs can be installed via package managers (`pip` for Python, `npm` for JavaScript/-TypeScript).

Chapter 2

Python SDK

2.1 Installation

Install the SYNAPSE Python SDK via pip:

```
pip install synapse-sdk
```

The SDK requires Python 3.8 or higher. For async support, install with the async extra:

```
pip install synapse-sdk[async]
```

2.2 Client Initialization

Initialize the SYNAPSE client with your API key:

```
from synapse import SynapseClient

# Initialize with API key
client = SynapseClient(api_key='your_api_key_here')

# Or initialize with explicit token (after obtaining via login)
client = SynapseClient(token='your_access_token')

# Optional: specify custom base URL or timeout
client = SynapseClient(
    api_key='your_api_key',
    base_url='https://api.synapse.ai/api/v1/',
    timeout=30
)
```

2.3 Authentication

2.3.1 Token-based Authentication

Authenticate using email and password to obtain a JWT token:

```
from synapse import SynapseClient

client = SynapseClient()
token = client.auth.login(
    email='user@example.com',
```

```
    password='secure_password'
)

# Token is automatically stored and used for subsequent requests
# Token will be automatically refreshed when expired
```

2.3.2 API Key Authentication

For server-to-server integrations, use API key authentication:

```
from synapse import SynapseClient

# Initialize with API key for seamless authentication
client = SynapseClient(api_key='sk_live_abc123xyz')

# All requests are automatically authenticated
response = client.articles.list()
```

2.3.3 Token Auto-Refresh

The SDK automatically handles token refresh:

```
from synapse import SynapseClient

client = SynapseClient(api_key='your_api_key')

# If token expires, SDK automatically refreshes it
# behind the scenes. No additional code needed.
for article in client.articles.list(page_size=100):
    print(article.title)
```

2.4 Articles API

2.4.1 List Articles

Retrieve paginated list of articles:

```
from synapse import SynapseClient

client = SynapseClient(api_key='your_api_key')

# List all articles
response = client.articles.list()
for article in response.results:
    print(f"{article.id}: {article.title}")

# Filter by topic
response = client.articles.list(topic='machine-learning')

# Pagination
response = client.articles.list(page=2, page_size=50)
print(f"Total articles: {response.total}")
print(f"Pages: {response.total_pages}")
```

2.4.2 Get Article

Retrieve a specific article by ID:

```
article = client.articles.get('article_id_123')
print(article.title)
print(article.content)
print(article.topic)
print(article.published_at)
print(article.source_url)
```

2.4.3 Search Articles

Search articles with semantic search:

```
# Semantic search (default)
results = client.articles.search(
    query='transformer models in NLP',
    semantic=True
)

# Keyword search
results = client.articles.search(
    query='attention mechanism',
    semantic=False
)

# Search with filters
results = client.articles.search(
    query='deep learning',
    semantic=True,
    topic='artificial-intelligence',
    min_score=0.7
)
```

2.4.4 Bookmark Article

Bookmark an article for later reference:

```
# Bookmark an article
client.articles.bookmark(article_id='article_123')

# List bookmarks
bookmarks = client.articles.list_bookmarks()

# Remove bookmark
client.articles.unbookmark(article_id='article_123')
```

2.5 Papers API

2.5.1 List Papers

Retrieve research papers:

```
# List all papers
papers = client.papers.list()

```

```python
# Filter by category
papers = client.papers.list(category='natural-language-processing')

# Filter by difficulty
papers = client.papers.list(difficulty='advanced')

# Combined filters
papers = client.papers.list(
    category='computer-vision',
    difficulty='intermediate'
)

# Pagination
papers = client.papers.list(page=1, page_size=20)
```

2.5.2 Get Paper

Retrieve a specific paper:

```python
paper = client.papers.get('paper_id_456')
print(paper.title)
print(paper.authors)
print(paper.abstract)
print(paper.pdf_url)
print(paper.published_date)
```

2.6 Repositories API

2.6.1 Trending Repositories

Get trending code repositories:

```python
# Trending repositories overall
repos = client.repos.trending()

# Filter by programming language
repos = client.repos.trending(language='python')

# Filter by time period
repos = client.repos.trending(period='weekly')  # daily, weekly, monthly

# Combined filters
repos = client.repos.trending(
    language='typescript',
    period='monthly'
)

# Iterate through results
for repo in repos.results:
    print(f"{repo.owner}/{repo.name}: {repo.stars} stars")
```

2.7 AI Chat API

2.7.1 Send Chat Message

Chat with SYNAPSE AI:

```python
# Simple chat
response = client.ai.chat(
    message='What are the latest advances in transformer models?'
)
print(response.reply)

# Conversation with context
response = client.ai.chat(
    message='Tell me more about attention mechanisms',
    conversation_id='conv_789'
)
```

2.7.2 Summarize Content

Generate summaries of content:

```python
# Summarize an article
summary = client.ai.summarize(
    content_type='article',
    content_id='article_123'
)
print(summary.summary_text)

# Summarize a paper
summary = client.ai.summarize(
    content_type='paper',
    content_id='paper_456'
)
```

2.8 Agents API

2.8.1 Create Task

Create an autonomous agent task:

```python
# Create a document generation task
task = client.agents.create_task(
    task_type='generate_document',
    prompt='Create a comprehensive guide on RAG systems',
    parameters={
        'document_type': 'pdf',
        'style': 'technical'
    }
)
print(f"Task ID: {task.id}")
print(f"Status: {task.status}")
```

2.8.2 Get Task Status

Retrieve task status and results:

```python
import time

task_id = 'task_abc123'

# Poll for task completion
```

```python
while True:
    task = client.agents.get_task(task_id)
    print(f"Status: {task.status}")

    if task.status == 'completed':
        print(f"Result: {task.result}")
        break
    elif task.status == 'failed':
        print(f"Error: {task.error}")
        break

    time.sleep(2)
```

2.8.3 Generate Document

Generate documents using AI agents:

```python
# Generate a PDF document
doc = client.agents.generate_document(
    doc_type='pdf',
    topic='Quantum Computing Fundamentals',
    prompt='Create a beginner friendly guide'
)

# Generate PowerPoint presentation
ppt = client.agents.generate_document(
    doc_type='pptx',
    topic='Latest AI Trends 2026',
    prompt='Include charts and key findings'
)

# Generate Word document
word = client.agents.generate_document(
    doc_type='docx',
    topic='Enterprise ML Deployment',
    prompt='Technical implementation guide'
)
```

2.9 Workflows API

2.9.1 Create Workflow

Create automation workflows:

```python
# Create a workflow
workflow = client.workflows.create(
    name='Daily AI News Digest',
    trigger='schedule',
    trigger_config={'cron': '0 9 * * *'},
    actions=[
        {
            'type': 'fetch_articles',
            'config': {'topic': 'artificial-intelligence', 'limit': 10}
        },
        {
            'type': 'summarize',
            'config': {'style': 'brief'}
```

```
        },
        {
            'type': 'send_email',
            'config': {'recipient': 'user@example.com'}
        }
    ]
)
```

2.9.2 List Workflows

Retrieve all workflows:

```
workflows = client.workflows.list()
for wf in workflows:
    print(f"{wf.name}: {wf.status}")
```

2.9.3 Trigger Workflow

Manually trigger a workflow:

```
result = client.workflows.trigger(workflow_id='workflow_123')
print(f"Execution ID: {result.execution_id}")
```

2.10 Trends API

2.10.1 Get Technology Radar

Retrieve technology trends and radar:

```
# Get overall technology radar
radar = client.trends.get_radar()

# Filter by category
radar = client.trends.get_radar(category='machine-learning')

# Iterate through trends
for trend in radar.trends:
    print(f"{trend.name}: {trend.momentum}")
```

2.10.2 Get Timeline

Get technology timeline:

```
# Get timeline for a technology
timeline = client.trends.get_timeline(
    technology='transformer-models',
    period='30d'  # 7d, 30d, 90d, 1y
)

for event in timeline.events:
    print(f"{event.date}: {event.description}")
```

2.11 Async Client

For high-concurrency applications, use the async client:

```
import asyncio
from synapse import SynapseAsyncClient

async def fetch_articles():
    async with SynapseAsyncClient(api_key='your_api_key') as client:
        articles = await client.articles.list(page_size=50)
        return articles

# Run async function
articles = asyncio.run(fetch_articles())
```

2.11.1 Concurrent Requests

Execute multiple concurrent requests:

```
import asyncio
from synapse import SynapseAsyncClient

async def concurrent_requests():
    async with SynapseAsyncClient(api_key='your_api_key') as client:
        # Execute multiple requests concurrently
        articles, papers, repos = await asyncio.gather(
            client.articles.list(),
            client.papers.list(),
            client.repos.trending()
        )
        return articles, papers, repos

result = asyncio.run(concurrent_requests())
```

2.12 Pagination Helper

Auto-paginate through large result sets:

```
from synapse import SynapseClient

client = SynapseClient(api_key='your_api_key')

# Auto-pagination generator
for article in client.articles.paginate(page_size=100):
    print(f"Processing: {article.title}")
    # Automatically fetches next page when exhausted
```

2.13 Error Handling

2.13.1 Exception Classes

The SDK provides specific exception classes:

```
from synapse import (
    SynapseAPIError,
    SynapseAuthError,
```

```
    SynapseRateLimitError,
    SynapseNotFoundError,
    SynapseValidationError
)

try:
    article = client.articles.get('nonexistent_id')
except SynapseNotFoundError as e:
    print(f"Article not found: {e}")
except SynapseAuthError as e:
    print(f"Authentication failed: {e}")
except SynapseRateLimitError as e:
    print(f"Rate limited. Retry after {e.retry_after} seconds")
except SynapseValidationError as e:
    print(f"Invalid parameters: {e}")
except SynapseAPIError as e:
    print(f"API error: {e}")
```

2.13.2 Retry Logic

SDK automatically retries on transient failures:

```
from synapse import SynapseClient

# Configure retry behavior
client = SynapseClient(
    api_key='your_api_key',
    max_retries=3,
    retry_backoff_factor=2.0  # exponential backoff
)

# Automatic retries on 429, 503, 504 errors
response = client.articles.list()
```

2.14 Webhook Handler

Handle webhooks from SYNAPSE:

```
from synapse import WebhookHandler
from flask import Flask, request

app = Flask(__name__)
webhook_handler = WebhookHandler(secret_key='your_webhook_secret')

@app.route('/webhooks/synapse', methods=['POST'])
def handle_webhook():
    try:
        event = webhook_handler.verify_and_parse(
            request.data,
            request.headers.get('X-Signature')
        )

        if event.type == 'article.created':
            print(f"New article: {event.data.title}")
        elif event.type == 'agent_task.completed':
            print(f"Task completed: {event.data.task_id}")

```

```
        return {'status': 'ok'}, 200
    except Exception as e:
        print(f"Webhook error: {e}")
        return {'error': str(e)}, 400

if __name__ == '__main__':
    app.run(port=5000)
```

Chapter 3

JavaScript/TypeScript SDK

3.1 Installation

Install the SYNAPSE JavaScript/TypeScript SDK via npm:

```
npm install @synapse-ai/sdk
```

Or with yarn:

```
yarn add @synapse-ai/sdk
```

For TypeScript projects, type definitions are included by default.

3.2 TypeScript Types

The SDK includes comprehensive TypeScript types:

```
// Article type
interface Article {
  id: string;
  title: string;
  content: string;
  topic: string;
  published_at: string;
  source_url: string;
  author: string;
  read_time_minutes: number;
}

// Paper type
interface Paper {
  id: string;
  title: string;
  authors: string[];
  abstract: string;
  category: string;
  difficulty: 'beginner' | 'intermediate' | 'advanced';
  pdf_url: string;
  published_date: string;
  citations: number;
}

// Repository type
interface Repository {
```

```
  id: string;
  owner: string;
  name: string;
  url: string;
  language: string;
  stars: number;
  forks: number;
  open_issues: number;
  description: string;
  last_updated: string;
}

// AgentTask type
interface AgentTask {
  id: string;
  task_type: string;
  status: 'pending' | 'processing' | 'completed' | 'failed';
  prompt: string;
  result?: any;
  error?: string;
  created_at: string;
  updated_at: string;
}

// Workflow type
interface Workflow {
  id: string;
  name: string;
  trigger: string;
  trigger_config: Record<string, any>;
  actions: WorkflowAction[];
  status: 'active' | 'inactive';
  created_at: string;
  updated_at: string;
}
```

3.3 Client Initialization

Initialize the SYNAPSE client:

```
import { SynapseClient } from '@synapse-ai/sdk';

// Initialize with API key
const client = new SynapseClient({
  apiKey: 'your_api_key_here'
});

// Or initialize with explicit token
const client = new SynapseClient({
  token: 'your_access_token'
});

// With custom configuration
const client = new SynapseClient({
  apiKey: 'your_api_key',
  baseURL: 'https://api.synapse.ai/api/v1/',
  timeout: 30000,
  maxRetries: 3
```

```
});
```

3.4 Articles API

3.4.1 List Articles

```
// List all articles
const articles = await client.articles.list();

// With pagination
const articles = await client.articles.list({
  page: 1,
  pageSize: 50
});

// Filter by topic
const articles = await client.articles.list({
  topic: 'machine-learning',
  page: 1,
  pageSize: 20
});

// Iterate results
articles.results.forEach((article) => {
  console.log(`${article.id}: ${article.title}`);
});
```

3.4.2 Get Article

```
const article = await client.articles.get('article_id_123');
console.log(article.title);
console.log(article.content);
```

3.4.3 Search Articles

```
// Semantic search
const results = await client.articles.search({
  query: 'transformer models in NLP',
  semantic: true
});

// Keyword search
const results = await client.articles.search({
  query: 'attention mechanism',
  semantic: false
});

// Search with filters
const results = await client.articles.search({
  query: 'deep learning',
  semantic: true,
  topic: 'artificial-intelligence',
  minScore: 0.7
});
```

3.4.4 Bookmark Article

```
// Bookmark
await client.articles.bookmark('article_id_123');

// List bookmarks
const bookmarks = await client.articles.listBookmarks();

// Remove bookmark
await client.articles.unbookmark('article_id_123');
```

3.5 Papers API

```
// List papers
const papers = await client.papers.list();

// Filter by category
const papers = await client.papers.list({
  category: 'natural-language-processing'
});

// Filter by difficulty
const papers = await client.papers.list({
  difficulty: 'advanced'
});

// Get single paper
const paper = await client.papers.get('paper_id_456');
```

3.6 Repositories API

```
// Trending repositories
const repos = await client.repos.trending();

// Filter by language
const repos = await client.repos.trending({
  language: 'python'
});

// Filter by period
const repos = await client.repos.trending({
  period: 'weekly'  // daily, weekly, monthly
});

// Combined filters
const repos = await client.repos.trending({
  language: 'typescript',
  period: 'monthly'
});
```

3.7 AI Chat API

```
// Simple chat
const response = await client.ai.chat({
  message: 'What are latest transformer advances?'
});
console.log(response.reply);

// Conversation with context
const response = await client.ai.chat({
  message: 'Tell me more about attention',
  conversationId: 'conv_789'
});

// Summarize content
const summary = await client.ai.summarize({
  contentType: 'article',
  contentId: 'article_123'
});
console.log(summary.summaryText);
```

3.8 Agents API

```
// Create task
const task = await client.agents.createTask({
  taskType: 'generate_document',
  prompt: 'Create guide on RAG systems',
  parameters: {
    documentType: 'pdf',
    style: 'technical'
  }
});

// Get task status
const task = await client.agents.getTask('task_abc123');

// Generate document
const doc = await client.agents.generateDocument({
  docType: 'pdf',
  topic: 'Quantum Computing',
  prompt: 'Create beginner guide'
});
```

3.9 Workflows API

```
// Create workflow
const workflow = await client.workflows.create({
  name: 'Daily AI News Digest',
  trigger: 'schedule',
  triggerConfig: { cron: '0 9 * * *' },
  actions: [
    {
      type: 'fetch_articles',
      config: { topic: 'ai', limit: 10 }
    },
    {
```

```
      type: 'summarize',
      config: { style: 'brief' }
    }
  ]
});

// List workflows
const workflows = await client.workflows.list();

// Trigger workflow
const result = await client.workflows.trigger('workflow_123');
```

3.10 Trends API

```
// Get technology radar
const radar = await client.trends.getRadar();

// Filter by category
const radar = await client.trends.getRadar({
  category: 'machine-learning'
});

// Get timeline
const timeline = await client.trends.getTimeline({
  technology: 'transformer-models',
  period: '30d'  // 7d, 30d, 90d, 1y
});
```

3.11 React Hooks

3.11.1 useSynapseArticles

```
import { useSynapseArticles } from '@synapse-ai/sdk/react';

export function ArticleList() {
  const { articles, loading, error } = useSynapseArticles({
    topic: 'machine-learning',
    pageSize: 20
  });

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error.message}</div>;

  return (
    <div>
      {articles.map((article) => (
        <div key={article.id}>
          <h3>{article.title}</h3>
          <p>{article.content.substring(0, 200)}...</p>
        </div>
      ))}
    </div>
  );
}
```

3.11.2 useSynapseChat

```
import { useSynapseChat } from '@synapse-ai/sdk/react';

export function ChatBox() {
  const { sendMessage, messages, loading } = useSynapseChat();

  const handleSend = async (message: string) => {
    const response = await sendMessage(message);
    console.log(response.reply);
  };

  return (
    <div>
      {messages.map((msg, idx) => (
        <div key={idx} className={msg.type}>
          {msg.text}
        </div>
      ))}
      <button
        onClick={() => handleSend('Hello')}
        disabled={loading}
      >
        Send
      </button>
    </div>
  );
}
```

3.11.3 useSynapseTrends

```
import { useSynapseTrends } from '@synapse-ai/sdk/react';

export function TrendsDashboard() {
  const { trends, loading } = useSynapseTrends({
    category: 'machine-learning'
  });

  if (loading) return <div>Loading trends...</div>;

  return (
    <ul>
      {trends.map((trend) => (
        <li key={trend.id}>
          {trend.name}: {trend.momentum}
        </li>
      ))}
    </ul>
  );
}
```

3.12 Error Handling

```
import {
  SynapseAPIError,
```

```
  SynapseAuthError,
  SynapseRateLimitError,
  SynapseNotFoundError
} from '@synapse-ai/sdk';

try {
  const article = await client.articles.get('invalid_id');
} catch (error) {
  if (error instanceof SynapseNotFoundError) {
    console.log('Article not found');
  } else if (error instanceof SynapseAuthError) {
    console.log('Auth failed');
  } else if (error instanceof SynapseRateLimitError) {
    console.log(`Rate limited. Retry after ${error.retryAfter}s`);
  } else if (error instanceof SynapseAPIError) {
    console.log(`API error: ${error.message}`);
  }
}
```

Chapter 4

Authentication Guide

4.1 Register and Get API Key

To obtain an API key for the SYNAPSE API:

1. Visit the SYNAPSE dashboard at `https://dashboard.synapse.ai`
2. Navigate to Settings > API Keys
3. Click "Create New Key"
4. Give your key a name (e.g., "Production API Key")
5. Select the appropriate permissions scope
6. Copy the generated key and store securely

API keys are long-lived and should be kept secret. Never commit keys to version control.

4.2 JWT Authentication Flow

The standard JWT authentication flow works as follows:

1. **Login**: Exchange credentials for access and refresh tokens
2. **Use**: Include access token in Authorization header
3. **Refresh**: When token expires, use refresh token to obtain new access token
4. **Repeat**: Continue using new access token

4.2.1 Login Endpoint

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "secure_password"
}
```

Response:

```
{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIs...",
    "refresh_token": "eyJhbGciOiJIUzI1NiIs...",
    "expires_in": 3600,
    "token_type": "Bearer"
  },
  "error": null,
  "meta": { ... }
}
```

4.2.2 Refresh Token Endpoint

```
POST /api/v1/auth/refresh
Content-Type: application/json

{
  "refresh_token": "eyJhbGciOiJIUzI1NiIs..."
}
```

4.2.3 Python JWT Example

```python
import requests
from datetime import datetime, timedelta

class JWTAuthManager:
    def __init__(self, email, password, base_url):
        self.email = email
        self.password = password
        self.base_url = base_url
        self.access_token = None
        self.refresh_token = None
        self.token_expires = None

    def login(self):
        response = requests.post(
            f'{self.base_url}/auth/login',
            json={'email': self.email, 'password': self.password}
        )
        data = response.json()['data']
        self.access_token = data['access_token']
        self.refresh_token = data['refresh_token']
        self.token_expires = datetime.now() + timedelta(
            seconds=data['expires_in']
        )

    def get_token(self):
        if self._is_expired():
            self._refresh()
        return self.access_token

    def _is_expired(self):
        return datetime.now() >= self.token_expires - timedelta(minutes=5)

```

```python
    def _refresh(self):
        response = requests.post(
            f'{self.base_url}/auth/refresh',
            json={'refresh_token': self.refresh_token}
        )
        data = response.json()['data']
        self.access_token = data['access_token']
        self.token_expires = datetime.now() + timedelta(
            seconds=data['expires_in']
        )
```

4.3 API Key Authentication

For server-to-server integrations, API key authentication is simpler:

```
Authorization: Bearer sk_live_abc123xyz
```

API keys are treated the same as JWT tokens in the Authorization header. They do not expire and do not require refresh.

4.3.1 Python API Key Example

```python
import requests

class APIKeyAuth:
    def __init__(self, api_key):
        self.api_key = api_key

    def get_headers(self):
        return {
            'Authorization': f'Bearer {self.api_key}',
            'Content-Type': 'application/json'
        }

auth = APIKeyAuth('sk_live_abc123xyz')

response = requests.get(
    'https://api.synapse.ai/api/v1/articles/',
    headers=auth.get_headers()
)
```

4.4 Token Storage Best Practices

- Store tokens in secure, encrypted storage
- Use environment variables for API keys
- Implement token rotation regularly
- Set appropriate token expiration times
- Never expose tokens in client-side code
- Use HTTPS for all API communication
- Implement secure logout that invalidates tokens
- Monitor token usage for suspicious patterns

4.4.1 Secure Token Storage (Python)

```python
import os
import json
from pathlib import Path
import keyring

class SecureTokenStore:
    def __init__(self, service_name='synapse-api'):
        self.service_name = service_name

    def save_token(self, token_key, token_value):
        # Use system keyring for storage
        keyring.set_password(self.service_name, token_key, token_value)

    def get_token(self, token_key):
        return keyring.get_password(self.service_name, token_key)

    def delete_token(self, token_key):
        keyring.delete_password(self.service_name, token_key)

store = SecureTokenStore()
store.save_token('access_token', token_value)
token = store.get_token('access_token')
```

4.5 Handling 401 Errors

A 401 error indicates authentication failure. Common causes:

- Missing or malformed Authorization header
- Token has expired
- Token is invalid or revoked
- Insufficient permissions

4.5.1 Automatic Token Refresh (Python)

```python
from synapse import SynapseClient

client = SynapseClient(api_key='sk_live_abc123xyz')

try:
    response = client.articles.list()
except SynapseAuthError as e:
    if e.status_code == 401:
        # Token expired, refresh and retry
        client.refresh_auth()
        response = client.articles.list()
    else:
        raise
```

Chapter 5

Webhooks

5.1 Available Webhook Events

SYNAPSE API supports the following webhook events:

Event Type	Description
article.created	Triggered when new article is published
article.updated	Triggered when article is updated
trend.updated	Triggered when technology trends change
agent_task.completed	Triggered when agent task completes
workflow.completed	Triggered when workflow execution finishes

5.2 Webhook Registration

Register a webhook endpoint via the API:

```
POST /api/v1/webhooks/
Content-Type: application/json
Authorization: Bearer YOUR_TOKEN

{
  "url": "https://your-domain.com/webhooks/synapse",
  "events": ["article.created", "trend.updated"],
  "active": true
}
```

Response:

```
{
  "success": true,
  "data": {
    "id": "webhook_abc123",
    "url": "https://your-domain.com/webhooks/synapse",
    "events": ["article.created", "trend.updated"],
    "active": true,
    "secret": "whsec_xyz789",
    "created_at": "2026-03-15T10:30:45Z"
  },
  "error": null,
  "meta": { ... }
}
```

5.3 Webhook Payload Format

Each webhook includes a standard payload:

```
{
  "id": "event_123abc",
  "type": "article.created",
  "timestamp": "2026-03-15T10:30:45Z",
  "data": {
    "article_id": "article_xyz",
    "title": "New AI Research Breakthrough",
    "topic": "artificial-intelligence"
  }
}
```

5.4 Webhook Signature Verification

All webhooks are signed with HMAC-SHA256. Verify signatures:

```
import hmac
import hashlib
import json

def verify_webhook_signature(payload, signature, secret):
    expected_signature = hmac.new(
        secret.encode(),
        payload.encode(),
        hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(signature, expected_signature)

# In webhook handler
payload = request.get_data(as_text=True)
signature = request.headers.get('X-Signature')
secret = 'whsec_xyz789'

if verify_webhook_signature(payload, signature, secret):
    event = json.loads(payload)
    process_event(event)
else:
    return {'error': 'Invalid signature'}, 403
```

5.5 Webhook Handler Example

5.5.1 FastAPI Webhook Handler

```
from fastapi import FastAPI, Request, HTTPException
import hmac
import hashlib

app = FastAPI()
WEBHOOK_SECRET = 'whsec_xyz789'

@app.post('/webhooks/synapse')
async def handle_webhook(request: Request):
    payload = await request.body()
```

```python
    signature = request.headers.get('X-Signature')

    # Verify signature
    expected_sig = hmac.new(
        WEBHOOK_SECRET.encode(),
        payload,
        hashlib.sha256
    ).hexdigest()

    if not hmac.compare_digest(signature, expected_sig):
        raise HTTPException(status_code=403, detail='Invalid signature')

    # Process event
    event = await request.json()

    if event['type'] == 'article.created':
        await handle_article_created(event['data'])
    elif event['type'] == 'agent_task.completed':
        await handle_task_completed(event['data'])

    return {'status': 'ok'}

async def handle_article_created(data):
    print(f"New article: {data['title']}")
    # Send notification, update database, etc.

async def handle_task_completed(data):
    print(f"Task completed: {data['task_id']}")
    # Process task result
```

5.6 Retry Policy

Failed webhook deliveries are retried with exponential backoff:

Attempt	Delay
1st attempt	Immediate
2nd attempt	5 minutes
3rd attempt	25 minutes
4th attempt	2 hours

Webhooks are considered failed if they return HTTP status >= 400 or timeout after 30 seconds. After 4 failed attempts, the webhook is disabled.

Chapter 6

Real-Time Updates

6.1 WebSocket Endpoint

Connect to SYNAPSE WebSocket for real-time updates:

```
wss://api.synapse.ai/ws/
```

WebSocket connections require authentication via token parameter in query string.

6.2 Server-Sent Events (SSE) Endpoint

For simple real-time updates without bidirectional communication:

```
GET /api/v1/stream/?token=YOUR_TOKEN
```

6.3 Available Channels

Subscribe to the following real-time channels:

Channel	Description
news_feed	Real-time new articles and papers
github_trends	Live GitHub trending repositories
agent_task_progress	Agent task status updates
ai_chat_stream	AI chat token-by-token responses
market_updates	Technology market insights

6.4 Python SSE Client Example

```python
import requests
import json

class SynapseSSEClient:
    def __init__(self, api_key, channel='news_feed'):
        self.api_key = api_key
        self.channel = channel
```

```python
        self.base_url = 'https://api.synapse.ai/api/v1'

    def subscribe(self, callback):
        headers = {'Authorization': f'Bearer {self.api_key}'}
        url = f'{self.base_url}/stream/?channel={self.channel}'

        response = requests.get(url, headers=headers, stream=True)

        for line in response.iter_lines():
            if line:
                try:
                    data = json.loads(line)
                    callback(data)
                except json.JSONDecodeError:
                    pass

# Usage
def on_article(data):
    print(f"New article: {data['title']}")

client = SynapseSSEClient(api_key='your_api_key')
client.subscribe(on_article)
```

6.5 JavaScript EventSource Example

```javascript
const token = 'your_access_token';
const eventSource = new EventSource(
  'https://api.synapse.ai/api/v1/stream/?channel=news_feed&token=' + token
);

eventSource.addEventListener('article.created', (event) => {
  const article = JSON.parse(event.data);
  console.log('New article:', article.title);
});

eventSource.addEventListener('error', (event) => {
  console.error('Connection error:', event);
  eventSource.close();
});
```

6.6 AI Chat Streaming (SSE)

Stream AI chat responses token by token:

```python
import requests
import json

def stream_chat(message, api_key):
    headers = {
        'Authorization': f'Bearer {api_key}',
        'Content-Type': 'application/json'
    }

    data = {
        'message': message,
```

```
        'stream': True
    }

    url = 'https://api.synapse.ai/api/v1/ai/chat/'
    response = requests.post(url, json=data, headers=headers, stream=True)

    full_response = ''
    for line in response.iter_lines():
        if line:
            try:
                chunk = json.loads(line)
                token = chunk.get('token', '')
                full_response += token
                print(token, end='', flush=True)
            except json.JSONDecodeError:
                pass

    return full_response

# Usage
response = stream_chat('Explain transformers', 'your_api_key')
```

Chapter 7

Rate Limiting

7.1 Rate Limit Headers

Every API response includes rate limit information:

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 987
X-RateLimit-Reset: 1647345645
X-RateLimit-Retry-After: 30
```

- `X-RateLimit-Limit`: Total requests allowed in current window
- `X-RateLimit-Remaining`: Requests remaining in current window
- `X-RateLimit-Reset`: Unix timestamp when limit resets
- `X-RateLimit-Retry-After`: Seconds to wait before retry (on 429)

7.2 Rate Limits by Endpoint

Different endpoints have different rate limits:

Endpoint	Limit (Pro)	Cost
/articles/	1000/min	1 req
/search/	500/min	5 req
/ai/chat/	100/min	10 req
/agents/tasks/	50/min	20 req
/webhooks/	100/min	1 req

7.3 Handling 429 Too Many Requests

When rate limited, implement exponential backoff:

```python
import time
import requests

def make_request_with_backoff(url, headers, max_retries=3):
    for attempt in range(max_retries):
        response = requests.get(url, headers=headers)

        if response.status_code == 429:
```

```
            retry_after = int(response.headers.get(
                'X-RateLimit-Retry-After',
                2 ** attempt
            ))
            print(f"Rate limited. Waiting {retry_after} seconds...")
            time.sleep(retry_after)
            continue

        return response

    raise Exception("Max retries exceeded")

# Usage
response = make_request_with_backoff(
    'https://api.synapse.ai/api/v1/articles/',
    headers={'Authorization': 'Bearer token'}
)
```

7.4 SDK Automatic Retry with Backoff

The Python SDK automatically handles rate limiting:

```
from synapse import SynapseClient

# SDK automatically retries on 429 with exponential backoff
client = SynapseClient(
    api_key='your_api_key',
    max_retries=5,
    retry_backoff_factor=2.0,
    retry_backoff_max=120  # Max 120 seconds between retries
)

# This will automatically retry if rate limited
articles = client.articles.list(page_size=100)
```

Chapter 8

Pagination

8.1 Standard Pagination Format

All list endpoints use consistent pagination:

```
{
  "success": true,
  "data": {
    "page": 1,
    "page_size": 20,
    "total": 5000,
    "total_pages": 250,
    "results": [
      { "id": "article_1", "title": "..." },
      { "id": "article_2", "title": "..." }
    ]
  },
  "error": null,
  "meta": { ... }
}
```

Fields:

- `page`: Current page number (1-indexed)
- `page_size`: Items per page (default: 20, max: 100)
- `total`: Total number of items
- `total_pages`: Total number of pages
- `results`: Array of items

8.2 Cursor-based Pagination for Real-Time Feeds

For high-frequency feeds, use cursor pagination:

```
GET /api/v1/articles/?cursor=abc123xyz&limit=50

{
  "success": true,
  "data": {
    "results": [ ... ],
    "next_cursor": "def456uvw",
```

```
    "has_more": true
  }
}
```

8.3 Python Pagination Helper

```python
from synapse import SynapseClient

client = SynapseClient(api_key='your_api_key')

# Manual pagination
page = 1
while True:
    response = client.articles.list(page=page, page_size=100)

    for article in response.results:
        process_article(article)

    if page >= response.total_pages:
        break

    page += 1

# Auto-pagination with generator
for article in client.articles.paginate(page_size=100):
    process_article(article)
    # Automatically fetches next page when exhausted
```

8.4 JavaScript Pagination

```javascript
async function fetchAllArticles() {
  const client = new SynapseClient({ apiKey: 'your_api_key' });
  let page = 1;
  let hasMore = true;

  while (hasMore) {
    const response = await client.articles.list({
      page: page,
      pageSize: 100
    });

    for (const article of response.results) {
      processArticle(article);
    }

    hasMore = page < response.total_pages;
    page++;
  }
}

// Or with async generator
async function* paginateArticles() {
  let page = 1;
  let hasMore = true;
```

```

  while (hasMore) {
    const response = await client.articles.list({ page });

    for (const item of response.results) {
      yield item;
    }

    hasMore = page < response.total_pages;
    page++;
  }
}

// Usage
for await (const article of paginateArticles()) {
  processArticle(article);
}
```

Chapter 9

Integration Examples

9.1 Example 1: Personal Tech Newsletter

Build a Python script that fetches articles, summarizes them with AI, and sends via email:

```python
import smtplib
from email.mime.text import MIMEText
from synapse import SynapseClient
from datetime import datetime, timedelta

class TechNewsletterGenerator:
    def __init__(self, api_key, email_config):
        self.client = SynapseClient(api_key=api_key)
        self.email_config = email_config

    def generate_newsletter(self, topic='artificial-intelligence'):
        # Fetch recent articles
        articles = self.client.articles.list(
            topic=topic,
            page_size=10
        )

        # Summarize each article
        summaries = []
        for article in articles.results:
            summary = self.client.ai.summarize(
                content_type='article',
                content_id=article.id
            )
            summaries.append({
                'title': article.title,
                'summary': summary.summary_text,
                'url': article.source_url
            })

        # Format as HTML
        html_content = self._format_newsletter(summaries)

        # Send email
        self._send_email(html_content)

    def _format_newsletter(self, summaries):
        html = '<h2>Tech Newsletter - ' + datetime.now().strftime(
            '%Y-%m-%d'
```

```
        ) + '</h2>'

        for item in summaries:
            html += f'''
            <div style="margin: 20px 0; border-left: 3px solid #0066cc;">
                <h3>{item['title']}</h3>
                <p>{item['summary']}</p>
                <a href="{item['url']}">Read More</a>
            </div>
            '''

        return html

    def _send_email(self, html_content):
        msg = MIMEText(html_content, 'html')
        msg['Subject'] = 'Tech Newsletter'
        msg['From'] = self.email_config['from']
        msg['To'] = self.email_config['to']

        with smtplib.SMTP(self.email_config['smtp_server']) as server:
            server.starttls()
            server.login(
                self.email_config['username'],
                self.email_config['password']
            )
            server.send_message(msg)

# Usage
config = {
    'from': 'newsletter@example.com',
    'to': 'user@example.com',
    'smtp_server': 'smtp.gmail.com',
    'username': 'your_email',
    'password': 'your_app_password'
}

generator = TechNewsletterGenerator('your_api_key', config)
generator.generate_newsletter(topic='machine-learning')
```

9.2 Example 2: Slack Bot for Tech Questions

Create a Slack bot that answers tech questions using SYNAPSE AI:

```
from slack_bolt import App
from synapse import SynapseClient

app = App(token='xoxb-your-token')
synapse_client = SynapseClient(api_key='your_api_key')

@app.message('^(?!.*<@).*')
def handle_message(message, say):
    user_message = message['text']

    # Search for relevant articles
    search_results = synapse_client.articles.search(
        query=user_message,
        semantic=True,
        page_size=5
```

```
    )

    # Get AI response
    ai_response = synapse_client.ai.chat(
        message=user_message
    )

    # Format response with sources
    response_text = f"*{ai_response.reply}*\n\n"
    response_text += "*Related Articles:*\n"

    for article in search_results.results[:3]:
        response_text += (
            f"- <{article.source_url}|{article.title}>\n"
        )

    say(response_text)

if __name__ == '__main__':
    app.start(port=3000)
```

9.3 Example 3: Automated Weekly AI Research Report

Generate weekly AI research reports and upload to Google Drive:

```
from synapse import SynapseClient
from google.oauth2 import service_account
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload
import tempfile

class ResearchReportGenerator:
    def __init__(self, api_key, google_creds):
        self.synapse = SynapseClient(api_key=api_key)
        self.drive_service = build(
            'drive', 'v3',
            credentials=service_account.Credentials.from_service_account_file(
                google_creds
            )
        )

    def generate_report(self):
        # Generate document using agents
        doc = self.synapse.agents.generate_document(
            doc_type='pdf',
            topic='Latest AI Research Trends 2026',
            prompt='Create a comprehensive research report with findings'
        )

        # Save to temporary file
        with tempfile.NamedTemporaryFile(
            suffix='.pdf',
            delete=False
        ) as tmp:
            tmp.write(doc.content)
            tmp_path = tmp.name

        # Upload to Google Drive
```

```
        file_metadata = {
            'name': f'AI_Research_Report_{datetime.now().date()}.pdf',
            'mimeType': 'application/pdf'
        }

        media = MediaFileUpload(
            tmp_path,
            mimetype='application/pdf'
        )

        self.drive_service.files().create(
            body=file_metadata,
            media_body=media,
            fields='id'
        ).execute()

        print(f"Report uploaded: {file_metadata['name']}")

# Usage
generator = ResearchReportGenerator(
    'your_api_key',
    'google_creds.json'
)
generator.generate_report()
```

9.4 Example 4: GitHub Actions Release Notes Generator

Use SYNAPSE API in GitHub Actions to generate release notes:

```
name: Generate Release Notes

on:
  release:
    types: [created]

jobs:
  release-notes:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2

      - name: Set up Python
        uses: actions/setup-python@v2
        with:
          python-version: '3.9'

      - name: Install dependencies
        run: |
          pip install synapse-sdk

      - name: Generate release notes
        env:
          SYNAPSE_API_KEY: ${{ secrets.SYNAPSE_API_KEY }}
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: python scripts/generate_release_notes.py
```

Python script for release notes:

```python
import os
from synapse import SynapseClient
import github

def generate_release_notes(repo_name, version):
    api_key = os.getenv('SYNAPSE_API_KEY')
    gh_token = os.getenv('GITHUB_TOKEN')

    synapse = SynapseClient(api_key=api_key)

    # Get trending repos and related articles
    trending = synapse.repos.trending(period='daily')

    # Get AI analysis
    analysis = synapse.ai.chat(
        message=f'Summarize key changes and improvements in {repo_name}
            v{version}'
    )

    # Create release notes
    notes = f'''
## Release Notes v{version}

{analysis.reply}

### Related Trending Projects
'''

    for repo in trending.results[:5]:
        notes += f'- [{repo.name}]({repo.url})\n'

    # Post to GitHub
    g = github.Github(gh_token)
    repo = g.get_repo(repo_name)
    release = repo.get_latest_release()
    release.edit(body=notes)

if __name__ == '__main__':
    generate_release_notes('owner/repo', '1.0.0')
```

9.5 Example 5: React Component for Trending Technologies

Display SYNAPSE trending technologies in a React component:

```typescript
import React, { useState, useEffect } from 'react';
import { useSynapseTrends } from '@synapse-ai/sdk/react';

export function TrendingTechnologies() {
  const [category, setCategory] = useState('machine-learning');
  const { trends, loading, error } = useSynapseTrends({
    category: category
  });

  const categories = [
    'machine-learning',
    'web-development',
    'devops',
```

```
    'cybersecurity',
    'data-engineering'
  ];

  if (loading) return <div className="spinner">Loading trends...</div>;
  if (error) return <div className="error">{error.message}</div>;

  return (
    <div className="trends-container">
      <h2>Trending Technologies</h2>

      <div className="category-selector">
        {categories.map((cat) => (
          <button
            key={cat}
            onClick={() => setCategory(cat)}
            className={category === cat ? 'active' : ''}
          >
            {cat}
          </button>
        ))}
      </div>

      <div className="trends-grid">
        {trends.map((trend) => (
          <div key={trend.id} className="trend-card">
            <h3>{trend.name}</h3>
            <div className="momentum-bar">
              <div
                className="momentum-fill"
                style={{width: `${trend.momentum * 100}%`}}
              />
            </div>
            <p className="momentum-text">{
              Math.round(trend.momentum * 100)
            }% momentum</p>
            <div className="trend-stats">
              <span>Mentions: {trend.mention_count}</span>
              <span>Growth: {trend.growth_rate}%</span>
            </div>
          </div>
        ))}
      </div>
    </div>
  );
}
```

Chapter 10

Semantic Search Guide

10.1 How Semantic Search Works

SYNAPSE semantic search uses embeddings and cosine similarity to find conceptually relevant content rather than exact keyword matches. The search engine:

1. Converts your query to a vector embedding
2. Compares against embeddings of all content
3. Returns results ranked by semantic similarity
4. Applies filters and re-ranks results

10.2 Semantic Search Endpoint

```
POST /api/v1/search/semantic/
Content-Type: application/json
Authorization: Bearer YOUR_TOKEN

{
  "query": "machine learning in healthcare",
  "limit": 20,
  "content_types": ["article", "paper"],
  "filters": {
    "topic": "healthcare",
    "min_score": 0.7,
    "published_after": "2025-01-01"
  }
}
```

10.3 Search Filters

Available search filters:

Filter	Description
content_type	article, paper, repo, video
topic	Technology topic (e.g., nlp, cv)
date_range	published_after, published_before
min_score	Minimum similarity score (0-1)
language	Content language (en, fr, de, etc.)
difficulty	beginner, intermediate, advanced

10.4 Search Result Ranking

Results are ranked by:

1. Semantic similarity score (cosine similarity)
2. Content freshness and recency
3. Engagement metrics (views, bookmarks)
4. Author credibility

10.5 Best Practices for Query Formulation

- Use natural language rather than keywords
- Be specific about your intent
- Include context when needed
- Avoid common stop words
- Use multiple searches if unsure

10.5.1 Examples

```python
from synapse import SynapseClient

client = SynapseClient(api_key='your_api_key')

# Good: Natural language query
results = client.articles.search(
    query='How do transformers improve NLP tasks',
    semantic=True
)

# Better: With context and filters
results = client.articles.search(
    query='practical applications of diffusion models in image generation',
    semantic=True,
    topic='computer-vision',
    min_score=0.75
)
```

```python

# Search across multiple content types
results = client.search(
    query='reinforcement learning for robotics',
    content_types=['article', 'paper', 'repo'],
    min_score=0.7
)
```

10.6 Cross-Content-Type Search

Search across articles, papers, repositories, and videos:

```python
results = client.search(
    query='graph neural networks applications',
    content_types=['article', 'paper', 'repo', 'video'],
    limit=50
)

articles = [r for r in results if r.type == 'article']
papers = [r for r in results if r.type == 'paper']
repos = [r for r in results if r.type == 'repo']
videos = [r for r in results if r.type == 'video']

for article in articles:
    print(f"Article: {article.title}")

for paper in papers:
    print(f"Paper: {paper.title} by {', '.join(paper.authors)}")

for repo in repos:
    print(f"Repo: {repo.owner}/{repo.name} ({repo.stars} stars)")
```

Chapter 11

AI Agent API Guide

11.1 Available Agent Task Types

SYNAPSE supports the following autonomous agent task types:

Task Type	Output	Description
generate_document	PDF/PPTX/DOCX	AI generates formatted document
analyze_trends	JSON report	AI analyzes technology trends
create_project	ZIP archive	Scaffolds coding project
research_topic	Markdown report	Comprehensive research summary
summarize_content	Text	Detailed content summary

11.2 Async Task Execution Pattern

Agent tasks execute asynchronously. The typical flow:

1. Create task (returns task ID immediately)
2. Poll task status until completion
3. Retrieve final result

11.2.1 Implementation

```python
import time
from synapse import SynapseClient

client = SynapseClient(api_key='your_api_key')

# Step 1: Create task
task = client.agents.create_task(
    task_type='generate_document',
    prompt='Create comprehensive guide on Transformer models',
    parameters={'document_type': 'pdf'}
)

task_id = task.id
print(f"Task created: {task_id}")
```

```python

# Step 2: Poll for completion
max_wait = 300  # 5 minutes
start_time = time.time()

while time.time() - start_time < max_wait:
    task = client.agents.get_task(task_id)
    print(f"Status: {task.status}")

    if task.status == 'completed':
        print("Task completed!")
        break
    elif task.status == 'failed':
        print(f"Task failed: {task.error}")
        break

    time.sleep(2)

# Step 3: Get result
if task.status == 'completed':
    result = task.result
    with open('guide.pdf', 'wb') as f:
        f.write(result)
```

11.3 Task Status Enum

Tasks progress through these states:

Status	Description
pending	Waiting to start execution
processing	Currently being executed
completed	Finished successfully
failed	Execution failed with error

11.4 WebSocket Real-Time Agent Progress

Monitor agent progress in real-time via WebSocket:

```python
import asyncio
import websockets
import json

async def monitor_task(task_id, api_key):
    url = f'wss://api.synapse.ai/ws/agent-progress/{task_id}?token={api_key}'

    async with websockets.connect(url) as websocket:
        async for message in websocket:
            event = json.loads(message)
            print(f"Status: {event['status']}")
            print(f"Progress: {event['progress']}%")

            if event['status'] == 'completed':
                print(f"Result: {event['result']}")
                break

# Usage
```

```
asyncio.run(monitor_task('task_abc123', 'your_api_key'))
```

11.5 Generate Document Task

```
# PDF document
pdf_doc = client.agents.generate_document(
    doc_type='pdf',
    topic='Quantum Computing Fundamentals',
    prompt='Create comprehensive beginner guide with diagrams'
)

# PowerPoint presentation
pptx_doc = client.agents.generate_document(
    doc_type='pptx',
    topic='2026 AI Trends',
    prompt='Include slide deck with key trends and statistics'
)

# Word document
word_doc = client.agents.generate_document(
    doc_type='docx',
    topic='Microservices Architecture',
    prompt='Technical guide for enterprise deployment'
)
```

11.6 Create Project Task

Generate code project scaffolds:

```
# Generate Python project
project = client.agents.create_task(
    task_type='create_project',
    prompt='Create FastAPI REST API for article management',
    parameters={
        'language': 'python',
        'framework': 'fastapi',
        'features': ['authentication', 'pagination', 'search']
    }
)

# Wait for completion...
task = client.agents.get_task(project.id)

# Download project as ZIP
with open('project.zip', 'wb') as f:
    f.write(task.result)
```

11.7 Analyze Trends Task

Get AI-powered trend analysis:

```
analysis_task = client.agents.create_task(
    task_type='analyze_trends',
    prompt='Analyze emerging trends in AI for 2026',
```

```
    parameters={
        'categories': ['machine-learning', 'nlp', 'robotics'],
        'depth': 'detailed',
        'include_predictions': True
    }
)

# Poll until complete
while True:
    task = client.agents.get_task(analysis_task.id)
    if task.status == 'completed':
        analysis = task.result
        print(f"Trend analysis: {analysis}")
        break
    time.sleep(5)
```

Chapter 12

API Changelog

12.1 Version 1.0.0 - March 2026

Initial release of SYNAPSE API v1.

12.1.1 Features

- REST API with JWT Bearer token authentication
- Articles, Papers, and Repositories endpoints
- Semantic search across content
- AI Chat and Content Summarization
- Autonomous Agent Task Execution
- Workflow Automation
- Technology Trends and Radar
- Real-time updates via WebSocket and SSE
- Webhook event notifications
- Rate limiting by subscription tier
- Comprehensive error handling and codes

12.1.2 SDKs

- Python SDK (synapse-sdk)
- JavaScript/TypeScript SDK (@synapse-ai/sdk)

12.1.3 Endpoints

Endpoint	Method	Description
/articles/	GET	List articles
/articles/id/	GET	Get single article
/articles/search/	POST	Semantic search
/papers/	GET	List papers
/papers/id/	GET	Get single paper
/repos/trending/	GET	Trending repositories
/ai/chat/	POST	Chat with AI
/ai/summarize/	POST	Summarize content
/agents/tasks/	POST	Create agent task
/agents/tasks/id/	GET	Get task status
/workflows/	GET, POST	List/create workflows
/trends/radar/	GET	Technology radar
/trends/timeline/	GET	Technology timeline
/webhooks/	POST	Register webhook
/search/semantic/	POST	Advanced semantic search

12.2 Versioning Policy

SYNAPSE API follows semantic versioning:

- **MAJOR**: Breaking changes (e.g., /api/v2/)
- **MINOR**: New features, backward compatible
- **PATCH**: Bug fixes, backward compatible

All versions are available at their respective endpoints (e.g., /api/v1/, /api/v2/). Breaking changes require a new major version. Clients should pin to a specific version.

12.3 Deprecation Policy

When deprecating API features:

1. Announce deprecation with 6 months notice
2. Return `Deprecation` header in responses
3. Provide migration guide
4. Support deprecated features for 12 months minimum
5. Remove after 12 month period with major version bump

12.3.1 Deprecation Example

When a feature is deprecated, responses include:

```
HTTP/1.1 200 OK
Deprecation: true
Sunset: Sun, 15 Mar 2027 00:00:00 GMT
Link: <https://docs.synapse.ai/migration/old-to-new>; rel="deprecation"

{
  "success": true,
```

```
  "data": { ... },
  "meta": {
    "warning": "This endpoint is deprecated. Use /api/v2/articles/ instead."
  }
}
```

12.4 Migration Guide Format

Migration guides follow this format:

1. Summary of changes
2. Old code example
3. New code example
4. Troubleshooting section
5. FAQs

12.4.1 Example Migration Guide

Migrating from v0.x to v1.0

Summary: Response format changed to include metadata envelope.

Old Format:

```
{
  "data": [ ... ],
  "total": 100
}
```

New Format:

```
{
  "success": true,
  "data": {
    "results": [ ... ],
    "total": 100
  },
  "error": null,
  "meta": { ... }
}
```

Migration Code:

```
# Old code (v0.x)
response = requests.get('https://api.synapse.ai/articles/')
articles = response.json()['data']

# New code (v1.0)
response = requests.get('https://api.synapse.ai/api/v1/articles/')
articles = response.json()['data']['results']
```

12.5 Future Versions Roadmap

12.5.1 v1.1 (Expected June 2026)

- Advanced filtering for search
- Batch operations endpoints
- Analytics dashboard API
- Team management features

12.5.2 v1.2 (Expected September 2026)

- Custom embedding models
- Advanced workflow triggers
- Webhook transformations
- Rate limit customization per endpoint

12.5.3 v2.0 (Expected March 2027)

- GraphQL API alongside REST
- Custom AI model fine-tuning
- Advanced access control (RBAC)
- Multi-tenant support
- Enhanced analytics and reporting

12.6 Support and Feedback

For API issues and questions:

- **Documentation**: https://docs.synapse.ai/api/
- **Status Page**: https://status.synapse.ai/
- **Support Email**: api-support@synapse.ai
- **Community Forum**: https://community.synapse.ai/
- **GitHub Issues**: https://github.com/synapse-ai/api-issues
- **Feature Requests**: https://feedback.synapse.ai/

12.7 Rate Limit Status

Check your current rate limit usage via headers:

```python
import requests

response = requests.get(
    'https://api.synapse.ai/api/v1/articles/',
    headers={'Authorization': 'Bearer token'}
)

print(f"Limit: {response.headers.get('X-RateLimit-Limit')}")
print(f"Remaining: {response.headers.get('X-RateLimit-Remaining')}")
print(f"Reset: {response.headers.get('X-RateLimit-Reset')}")
```

12.8 API Health and Monitoring

Monitor API health and uptime:

```
GET https://api.synapse.ai/health/

{
  "status": "healthy",
  "version": "1.0.0",
  "timestamp": "2026-03-15T10:30:45Z",
  "services": {
    "database": "healthy",
    "cache": "healthy",
    "search": "healthy",
    "ai_engine": "healthy"
  }
}
```

12.9 API Performance Metrics

Expected API performance characteristics:

Metric	Target
Average Response Time	¡ 200ms
P95 Response Time	¡ 500ms
P99 Response Time	¡ 1000ms
Availability	99.9%
Cache Hit Rate	¿ 80%
Search Query Time	¡ 500ms
AI Response Time	2-10s

12.10 Best Practices Summary

12.10.1 API Usage

- Use connection pooling for multiple requests
- Implement exponential backoff for retries
- Cache responses when appropriate
- Use pagination for large result sets

- Monitor rate limit headers
- Implement proper error handling
- Use webhooks for event notifications
- Use semantic search for better results

12.10.2 Security

- Never expose API keys in client-side code
- Store keys in secure environment variables
- Use HTTPS for all requests
- Implement API key rotation
- Monitor token usage
- Verify webhook signatures
- Implement request signing for critical operations

12.10.3 Performance

- Batch API requests when possible
- Use async clients for concurrent operations
- Enable response compression
- Cache frequently accessed data
- Use cursor-based pagination for large datasets
- Implement request deduplication
- Monitor and optimize slow queries

Chapter 13

Appendix: Complete API Reference

13.1 Environment Variables

Standard environment variables for SYNAPSE SDK:

```
SYNAPSE_API_KEY=sk_live_abc123xyz
SYNAPSE_API_BASE_URL=https://api.synapse.ai/api/v1/
SYNAPSE_TIMEOUT=30
SYNAPSE_MAX_RETRIES=3
SYNAPSE_DEBUG=false
```

13.2 Common Response Codes

Code	Status	Meaning
200	OK	Request succeeded
201	Created	Resource created
204	No Content	Success, no response body
400	Bad Request	Invalid request
401	Unauthorized	Invalid credentials
403	Forbidden	No permission
404	Not Found	Resource not found
409	Conflict	Resource conflict
429	Too Many Requests	Rate limited
500	Internal Error	Server error
502	Bad Gateway	Service temporarily unavailable
503	Service Unavailable	Maintenance or overload

13.3 Request Header Reference

Standard headers to include with requests:

```
Authorization: Bearer YOUR_TOKEN
Content-Type: application/json
User-Agent: YourApp/1.0
X-Request-ID: unique-request-id
Accept: application/json
```

13.4 Response Header Reference

Standard headers in API responses:

```
Content-Type: application/json
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 987
X-RateLimit-Reset: 1647345645
X-Request-ID: req_abc123
Cache-Control: public, max-age=300
ETag: "abc123xyz"
```

13.5 Useful Resources

- Official API Documentation: https://docs.synapse.ai/
- API Explorer (Interactive): https://api.synapse.ai/explore/
- SDK Documentation: https://sdk.synapse.ai/
- GitHub Repository: https://github.com/synapse-ai/sdk
- Stack Overflow Tag: synapse-api
- Community Slack: https://slack.synapse.ai/
- Blog and Tutorials: https://blog.synapse.ai/

13.6 Code Examples Repository

Complete working examples available at:

```
https://github.com/synapse-ai/api-examples/

Structure:
  python/
    - newsletter_generator.py
    - slack_bot.py
    - research_automation.py
  javascript/
    - react_components.js
    - node_examples.js
  workflows/
    - github_actions_example.yml
```